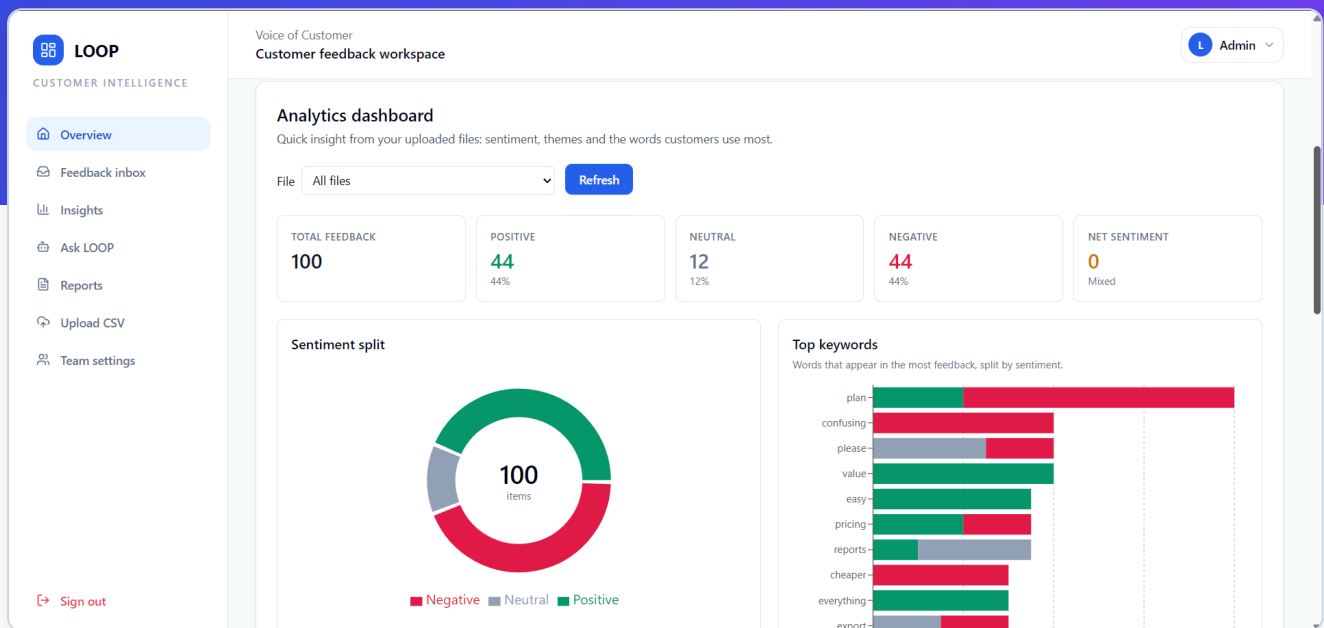




Project LOOP

Turn customer feedback files into sentiment, themes and answers, in minutes.

PROJECT REPORT



Developers	Sonia and Purnima
Live application	https://project-loop-two.vercel.app
Source code	https://github.com/devxjitin/Project-loop
Report date	20 September 2026

Next.js + React

FastAPI

PostgreSQL + pgvector

Google Gemini

BullMQ + Redis

Vercel

Render

Neon

Executive summary

LOOP is a web application that turns a spreadsheet of customer feedback into clear, trustworthy insight. Upload a CSV of reviews, survey answers or support comments, and LOOP labels every item as positive, neutral or negative, groups similar feedback into named themes, shows what customers talk about most, and lets you ask questions in plain language.

The problem

Teams collect feedback in spreadsheets from surveys, app stores and support desks. Reading thousands of rows by hand is slow, and summaries tend to reflect whatever the reader noticed first. Generic AI chat tools can summarise text, but they can invent details and give no way to check where an answer came from.

The solution

LOOP combines an automated analysis pipeline with a dashboard and a question-answering assistant. Every answer names the uploaded file and the exact feedback it used, and when the data does not support an answer LOOP says so instead of guessing.

Objectives

- **Fast insight from a single file:** analyse one CSV on its own, or compare all uploaded files together.
- **Trustworthy AI:** every claim carries a citation, and unsupported answers are rejected.
- **Safe multi-tenant design:** each workspace sees only its own data, enforced inside the database.
- **Low running cost:** the whole system runs on free hosting tiers.
- **Clear feedback while working:** live progress and plain messages when free services are waking up.

3

hosting platforms
Vercel, Render,
Neon

2

external APIs
Google Gemini,
Resend

5

background workers

18

database migrations

3

user roles
admin, editor,
viewer

50 MB

maximum CSV
upload

Contents

Product tour: sign up, sign in, upload and analytics	3
Features and user roles	5
System architecture	6
Workflow: from upload to insight	7
Ask LOOP: grounded question answering	8
Data model and tenant isolation	9
Technology stack	10
Deployment and CI/CD	11
Security, privacy and quality	12
Challenges, lessons learned and next steps	13
Team, links and API reference	14

Getting started: create a workspace and sign in

Every team works inside its own private workspace. The first person to sign up becomes the workspace admin and can then invite teammates by email.

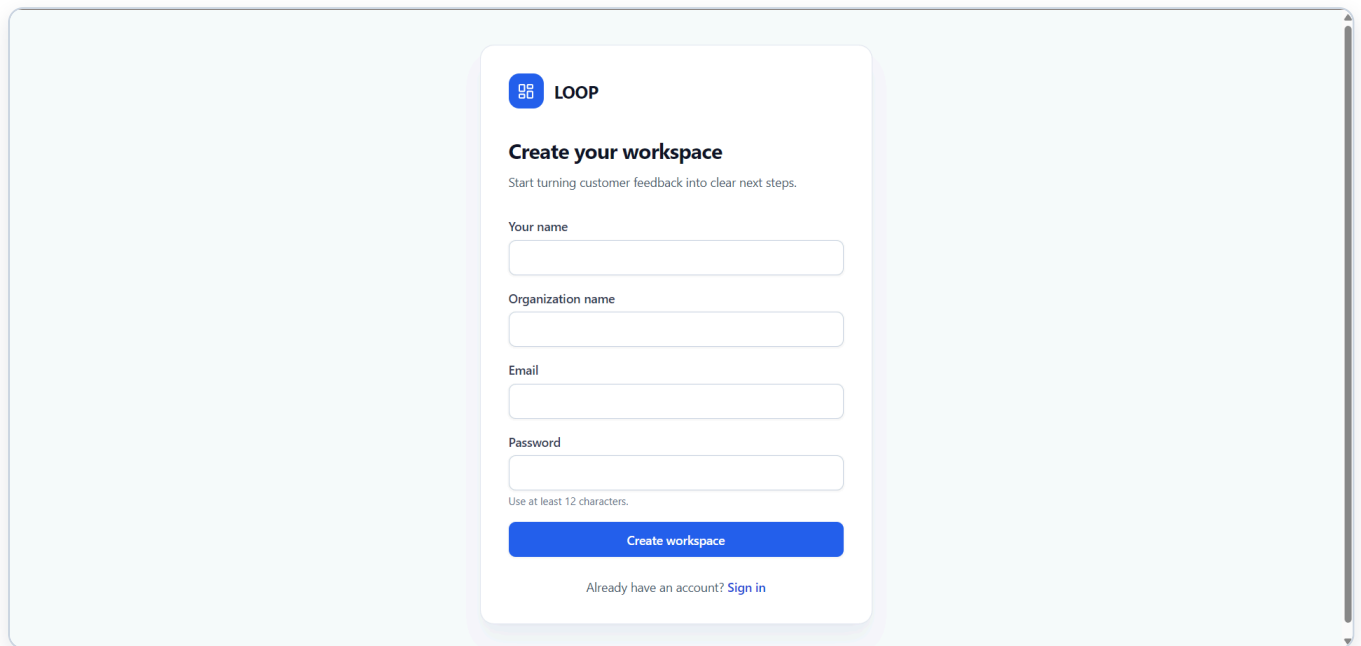
The screenshot shows a web form titled "LOOP" with the heading "Create your workspace". Below the heading is a subtext: "Start turning customer feedback into clear next steps." The form contains four input fields: "Your name", "Organization name", "Email", and "Password". Below the "Password" field is a note: "Use at least 12 characters." At the bottom of the form is a blue button labeled "Create workspace". Below the button is a link: "Already have an account? Sign in".

Figure 1. Create your workspace. New users enter a name, an organization name, an email and a password of at least 12 characters. Creating a workspace also creates its first admin account.

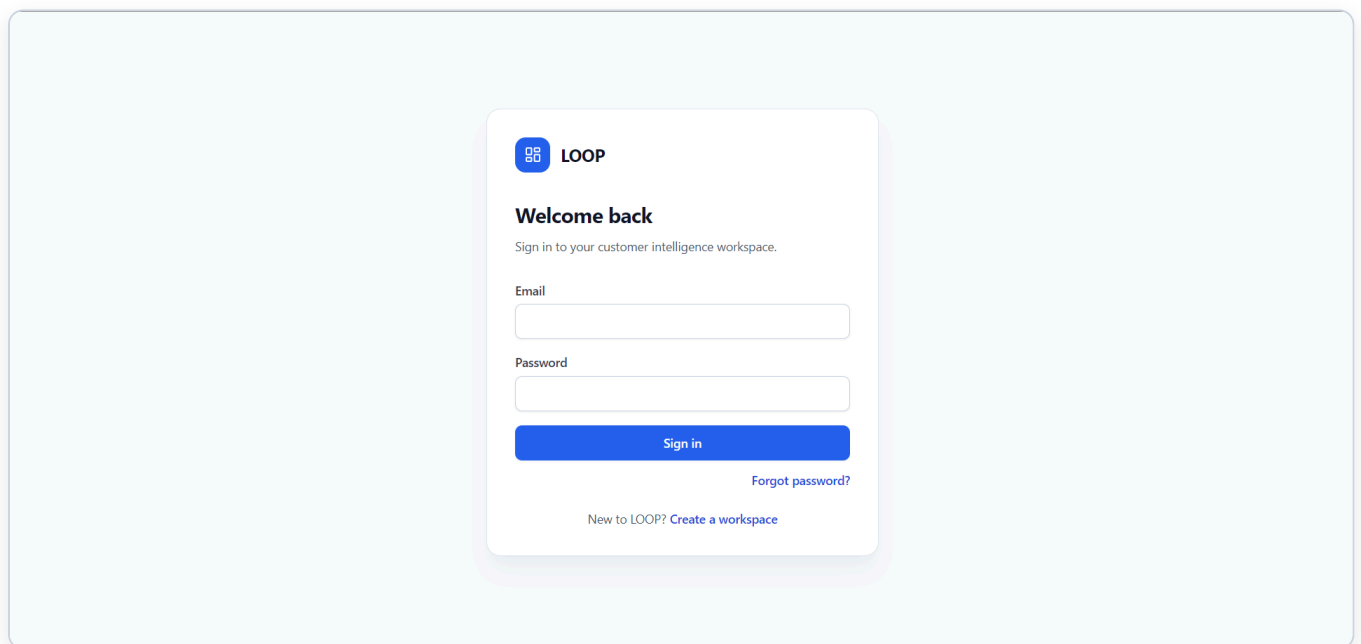
The screenshot shows a web form titled "LOOP" with the heading "Welcome back". Below the heading is a subtext: "Sign in to your customer intelligence workspace." The form contains two input fields: "Email" and "Password". Below the "Password" field is a blue button labeled "Sign in". Below the button are two links: "Forgot password?" and "New to LOOP? Create a workspace".

Figure 2. Sign in. Returning users sign in with email and password. A "Forgot password?" link sends a reset email, and "Create a workspace" leads back to sign-up. Sessions use short-lived signed tokens.

Private by design. Each workspace is isolated inside the database. Creating a workspace never gives access to another team's data, and only admins decide who is invited and what role they get.

Upload a dataset and read the analytics

Voice of Customer
Customer feedback workspace

ADMIN

DATASET UPLOAD

Add customer feedback from any CSV.

Rename the dataset and select the CSV column that contains the main customer reviews.

Upload customer dataset

Choose any CSV, rename the dataset, then select the column containing the main customer reviews.

Choose a CSV file
CSV only, maximum 50 MB

Dataset name

Main review column
Select a column

Date fields are not imported.

Recent uploads

DATASET	STATUS
	UPLOADED

Figure 3. Dataset upload. An admin chooses a CSV file (up to 50 MB), gives the dataset a name and selects the column that holds the main customer reviews. Only that column is imported, and date fields are not imported. The *Recent uploads* list below the form shows each file and its status, and deleting a file also deletes the feedback and analytics that came from it.

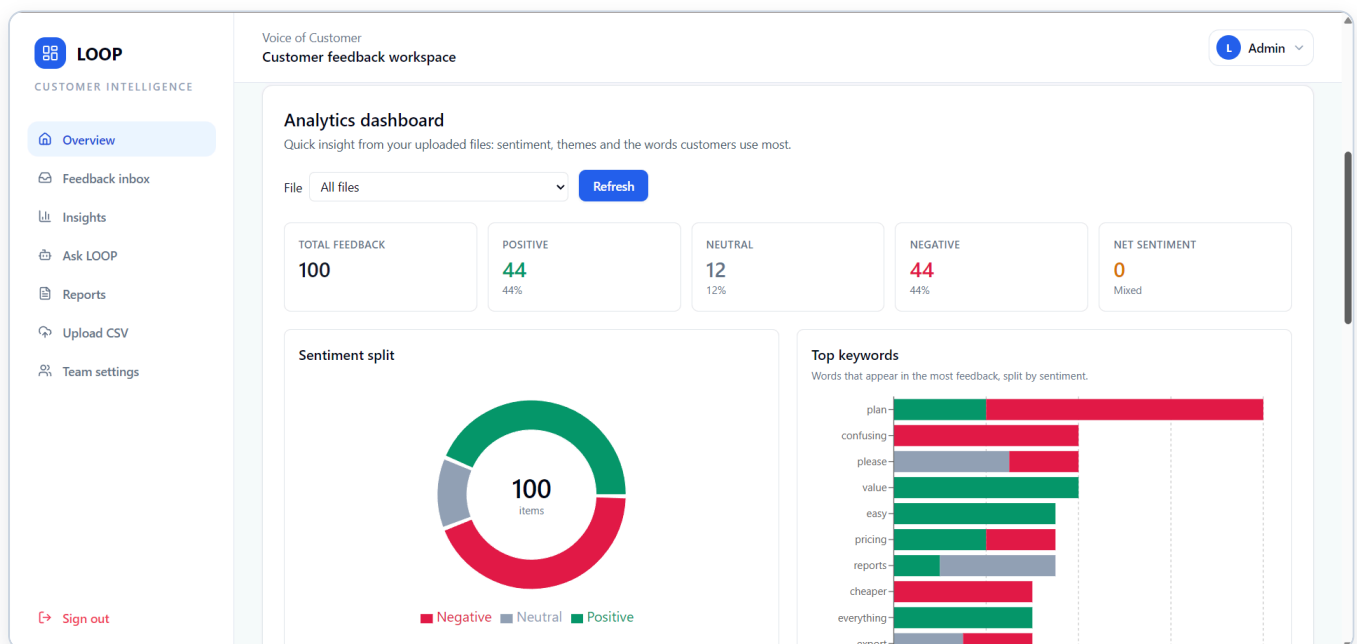


Figure 4. Analytics dashboard. Summary cards show total feedback, positive, neutral and negative counts with percentages, and a net sentiment score. The sentiment split donut and the top keywords chart (each word split by sentiment) sit below. In this sample run a 100-row file produced 44 positive, 12 neutral and 44 negative items. Further down the page the dashboard shows top themes, biggest pain points and strengths, a per-file comparison and, when the data spans several days, a trend over time. A **File** dropdown switches between all files and a single file.

Features and user roles

Feature	What you get
CSV upload	Upload a file, name the dataset and pick the feedback column. Rows are imported in bulk, so a large file loads in seconds.
Sentiment analysis	Every item is labelled positive, neutral or negative by Google Gemini. Results are cached by a hash of the text, so identical feedback is classified only once.
Themes	Feedback is embedded as vectors and clustered by meaning with HDBSCAN. Each cluster is named in plain language and reused on later runs when it is at least 0.90 similar.
Analytics dashboard	Sentiment split, net sentiment score, top themes, pain points and strengths, top keywords, per-file comparison and a time trend when relevant.
File-wise analytics	View all files together or one file alone. Deleting a file removes its feedback, themes and numbers immediately.
Feedback inbox	Search and filter by file, sentiment and theme, and open any item for its full text and file name.
Ask LOOP	Ask questions in plain language. Answers use only your feedback, cite the file and item behind every claim, and say "I don't know" when the data does not support an answer.
Reports	Generate an executive summary on demand or on a weekly or monthly schedule, share it by an expiring link, or download it as a PDF.
Team management	Invite teammates by email, assign roles and remove members. Password reset is built in.

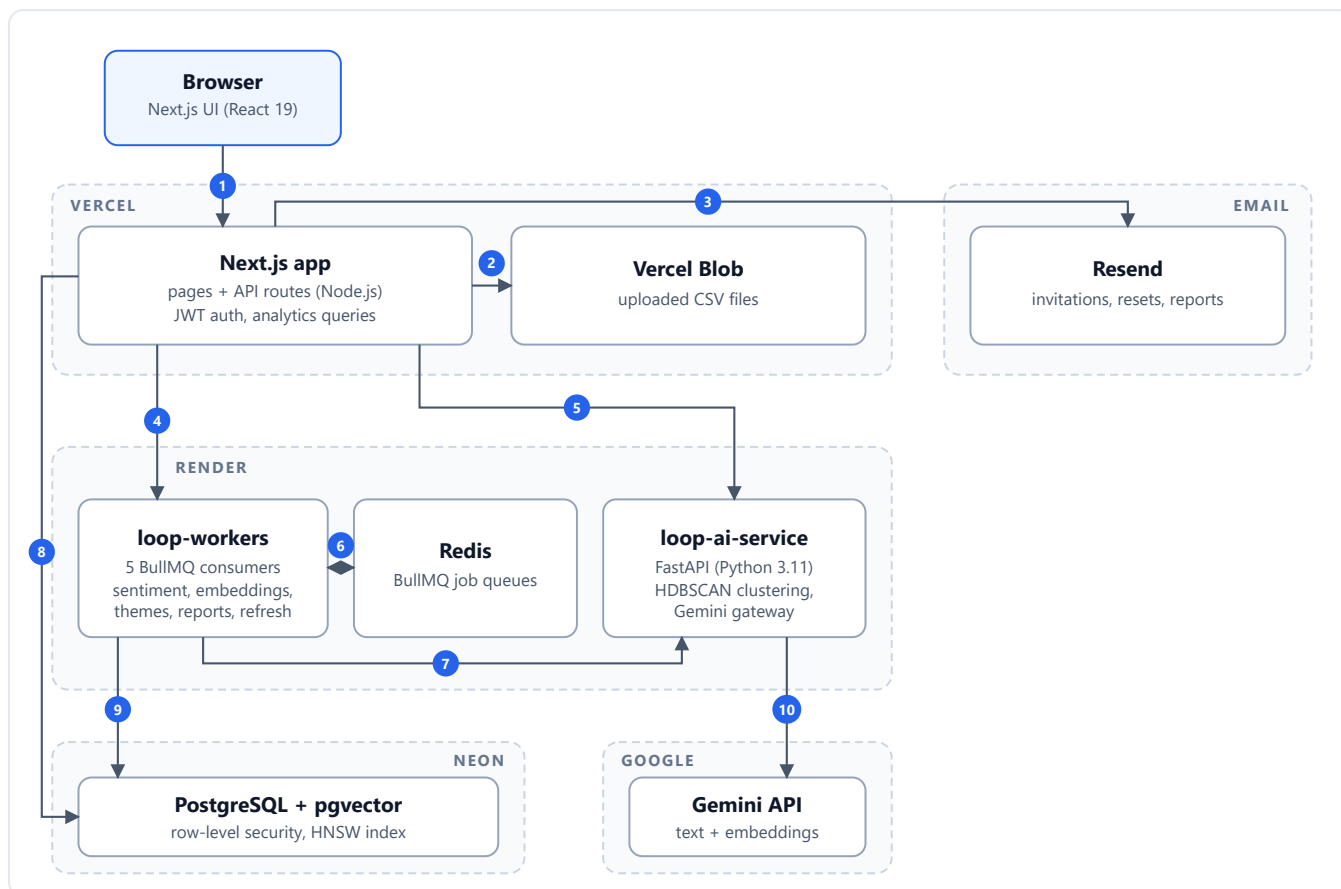
User roles

Role	What they can do
Admin	Everything: create and delete uploads, generate and share reports, set report schedules, and manage team members and roles.
Editor	Read access to insights, the inbox and Ask LOOP, and permission to send file data into an upload started by an admin.
Viewer	Read-only access to insights, the inbox and Ask LOOP.

Design principle. LOOP is built for quick analysis based on files. It does not depend on any date range, and each file can be studied on its own or alongside the others.

System architecture

LOOP is split into a web tier, a background-processing tier and a data tier. Each runs on the platform best suited to it, and every one has a free plan.



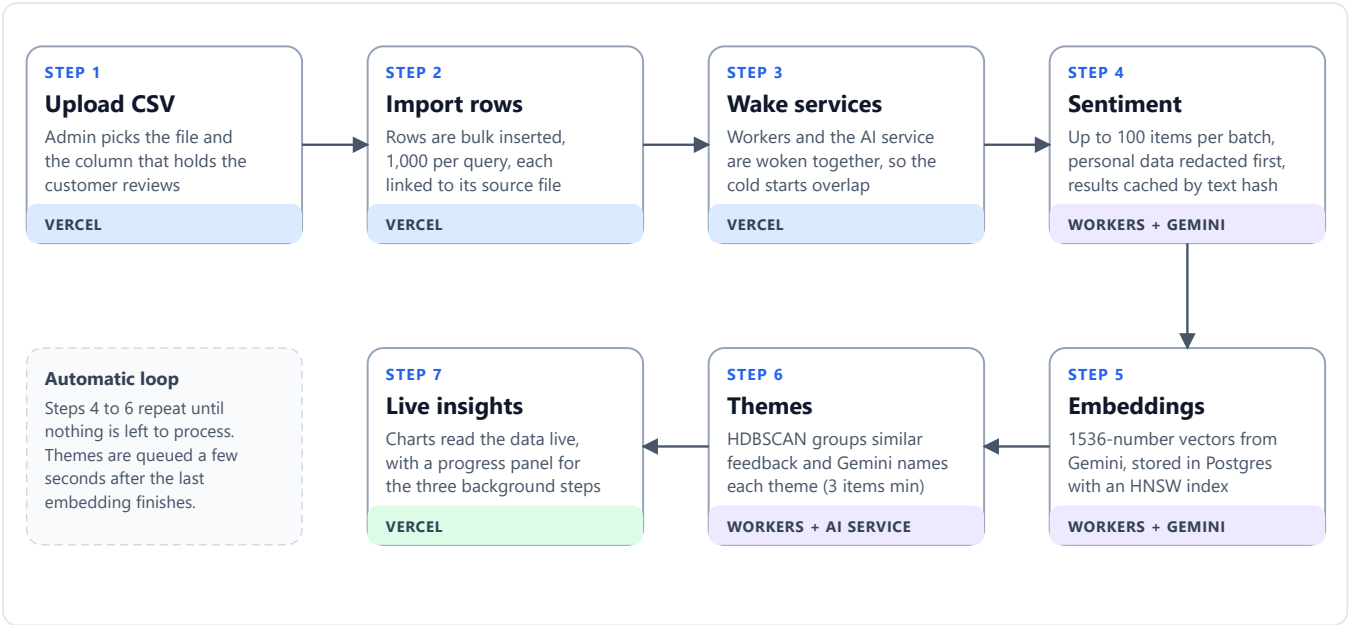
- 1 Browser calls the Next.js app over HTTPS.
- 2 Uploaded CSV files are stored in Vercel Blob.
- 3 Emails (invites, resets, reports) go out through Resend.
- 4 After an upload the app wakes the sleeping workers.
- 5 Questions and report summaries call the AI service.
- 6 Workers take jobs from the Redis queues.
- 7 Workers send batches to the AI service.
- 8 The web app reads and writes as the restricted `loop_app` database role.
- 9 Workers use the privileged `loop_worker` role.
- 10 The AI service calls Google Gemini.

Key design decisions

- **Separate AI service:** keeps the Gemini key and the heavy clustering libraries out of the web tier.
- **Queue workers:** analysing a large file takes minutes, so queues with retries keep the web app responsive.
- **Postgres with pgvector:** one database holds the relational data and the vectors, so tenant isolation covers both.
- **Live analytics queries:** there are no cached copies to go stale, and deleting a file updates every chart at once.
- **Free tiers on purpose:** Vercel, Render and Neon free plans keep the running cost at zero, at the price of cold starts.

From upload to insight

Uploading a file starts an automatic pipeline. The dashboard shows live progress for each stage and fills in the charts as results arrive.



What each stage does

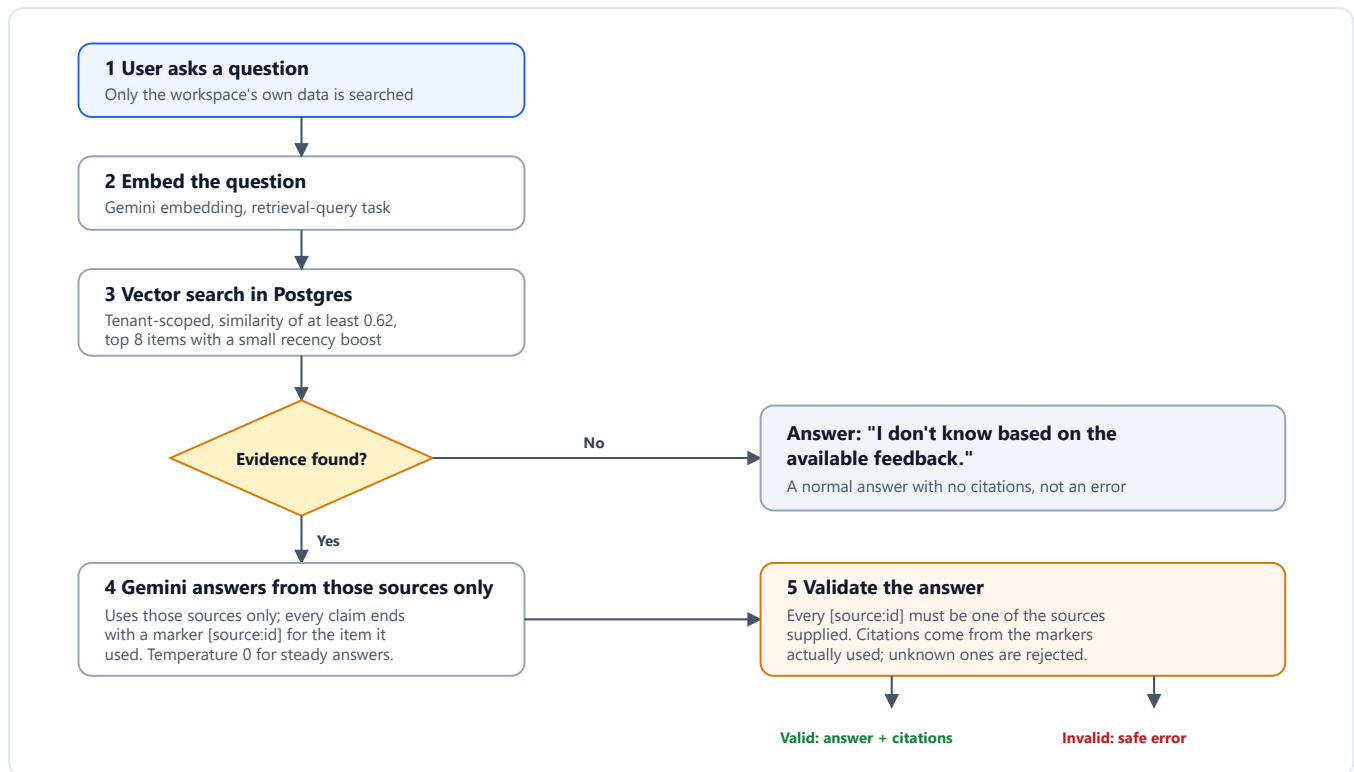
Stage	Details
Import	The selected column is parsed and inserted in chunks of 1,000 rows. External IDs make the import repeatable, and each row keeps a link to its source file.
Sentiment	The worker sends up to 100 unique messages per request. Emails, phone numbers, card numbers and labelled personal fields are redacted first. Gemini returns exactly one label per item, and the service rejects incomplete or duplicated answers.
Embeddings	Each item becomes a 1536-number vector with <code>gemini-embedding-001</code> , stored with an HNSW cosine index for fast similarity search.
Themes	HDBSCAN groups vectors that are close in meaning. Gemini names each cluster. A theme is reused when its centre is at least 0.90 similar to an existing one, so names stay stable.
Insights	Charts read the feedback table live. Deleting a file removes its numbers at once. The status endpoint reports progress for the three background stages.

Reliability behaviours

- **Cold starts:** free services sleep when idle. The web app wakes the workers and AI service together after an upload, and workers retry a 502, 503 or 504 answer for about ninety seconds.
- **Rate limits:** after a Gemini rate limit, workers pause 45 seconds and continue instead of failing repeatedly.
- **Honest progress:** if nothing has moved for 45 seconds, the panel explains that a free service is waking up or waiting out a rate limit.

Ask LOOP: grounded question answering

Ask LOOP answers questions such as "What do customers complain about most?" using only your own feedback. It is designed so that every claim can be checked.



Why it can be trusted

- **Grounded by construction:** the model only sees the retrieved feedback and is told not to use outside knowledge.
- **Checkable:** each citation names the uploaded file and opens the matching item in the feedback inbox.
- **Honest uncertainty:** when nothing relevant is found, LOOP says it does not know instead of inventing an answer.
- **Private:** retrieval is limited to the workspace by both a database policy and an explicit tenant filter.

Tested behaviour. With complaint-heavy sources, a broad question such as "what is the biggest problem in the product" returned a real answer citing 8 items in 3 of 3 tries. With unrelated questions it returned a clean "I don't know" every time.

Example

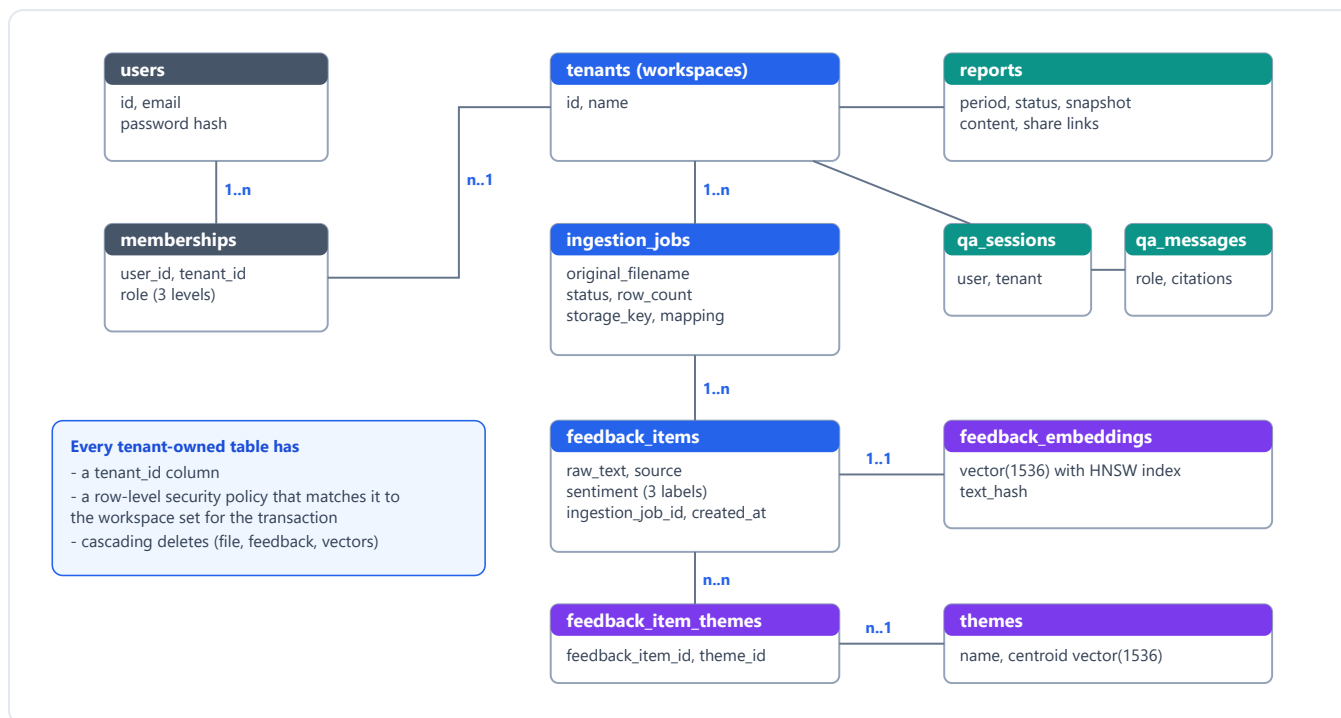
Question: "what is the biggest problem occur in product"

Answer (shortened): "Based on the provided feedback, the most recurring problems mentioned by customers involve team permissions being confusing, poorly explained, failing, and not working properly."

Test run against the sample dataset. The full answer cited 8 feedback items, each marked with its source and linked to the inbox.

Data model and tenant isolation

Every workspace is a *tenant*. All tenant-owned tables carry a `tenant_id` and a row-level security policy, so the database itself refuses to return another workspace's rows.



Two database roles

loop_app (web app)

Used by the Vercel-hosted API. It cannot bypass row-level security, so a query can only ever see the signed-in workspace. Each request sets the workspace at the start of its transaction.

loop_worker (background workers)

Used only by the Render workers, which process many workspaces. It bypasses row-level security, so every worker query also filters by `tenant_id` explicitly. The web app is never given this credential.

Technology stack

Layer	Technology	Why it is used
Frontend	Next.js 16 (App Router), React 19, TypeScript, Tailwind CSS 3	Server and client rendering in one project, type safety and fast styling.
Charts and icons	Recharts, lucide-react	Donut, bar and area charts; consistent icons.
Backend API	Next.js route handlers (Node.js runtime), JWT, bcrypt	One deployable for pages and API; signed sessions and hashed passwords.
Background jobs	BullMQ on Redis	Reliable queues with retries and backoff for sentiment, embeddings, themes, reports and refresh jobs.
AI service	Python 3.11, FastAPI, Uvicorn, HDBSCAN, scikit-learn, NumPy	Keeps AI keys and clustering off the web tier; Python has the best clustering libraries.
AI models	Google Gemini: <code>gemini-3.5-flash-lite</code> for text, <code>gemini-embedding-001</code> at 1536 dimensions	Sentiment, cluster names, answers and report summaries; vectors for similarity search.
Database	PostgreSQL 16+ with pgvector (HNSW cosine index)	Relational data and vector search together, with row-level security for isolation.
File storage	Vercel Blob (local disk in development)	Private storage for the uploaded CSV files.
Email	Resend	Invitations, password resets and report delivery.
PDF reports	Headless Chromium	Renders the same report page users see into a PDF.
Testing and quality	Node test runner, ESLint, TypeScript, Ruff, mypy, gitleaks	Integration tests on a real database, static checks and secret scanning.
CI/CD	GitHub Actions, Vercel CLI, Render Docker builds	Automatic checks on every push and repeatable deploys.
Local development	Docker Compose	The full stack (database, Redis, AI service, web, workers) with one command.

Hosting platforms

Vercel

Web app, API routes and Blob file storage.

Render

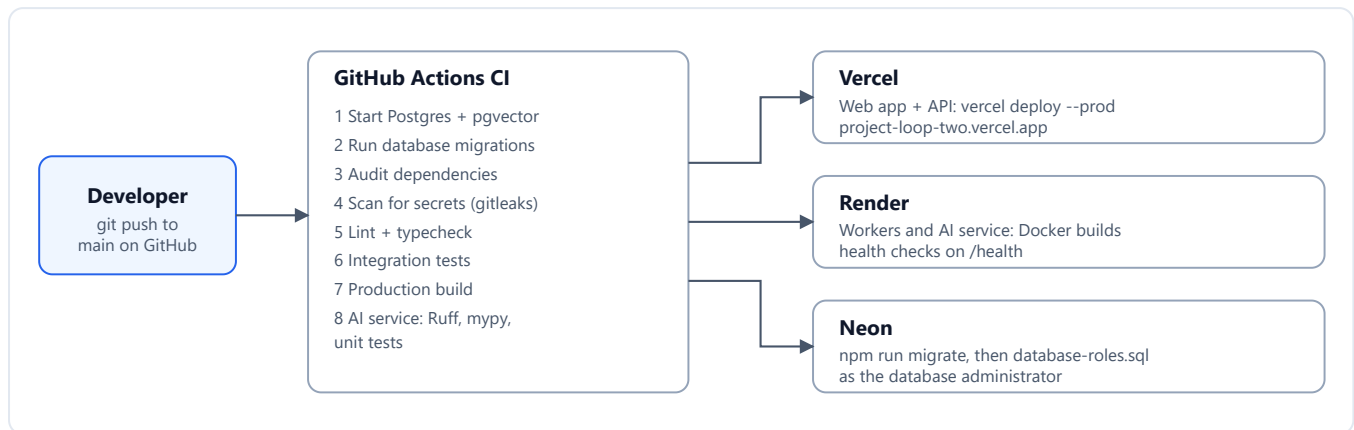
AI service, background workers and the Redis queue store, on free plans.

Neon

Managed PostgreSQL with pgvector.

External APIs: Google Gemini for AI and Resend for email.

Deployment and CI/CD



Environments and live endpoints

Component	Platform	Deploy method and health check
Web app and API	Vercel	Vercel CLI. Health: https://project-loop-two.vercel.app/api/health
AI service	Render	Docker image from <code>apps/ai-service</code> . Health: https://loop-ai-service-801c.onrender.com/health
Background workers	Render	Docker image from <code>apps/web</code> , one process runs all five consumers. Health: https://loop-workers.onrender.com/health
Queues	Render	Managed Redis (Key Value) used by BullMQ.
Database	Neon	Managed Postgres; migrations run from the repository.

Configuration

Secrets are stored only in each platform's environment settings and never in the repository. The main variables are the two database URLs (`DATABASE_URL` for the web app, `DATABASE_WORKER_URL` for workers only), `REDIS_URL`, `AI_SERVICE_URL` with a shared `AI_SERVICE_TOKEN`, `GEMINI_API_KEY`, `JWT_SECRET`, `BLOB_READ_WRITE_TOKEN`, `RESEND_API_KEY`, `EMAIL_FROM`, `APP_URL` and `WORKER_WAKE_URL`.

Free-plan behaviour. Render's free services sleep after about 15 minutes without traffic and take 30 to 60 seconds to wake. LOOP softens this by waking services together and by retrying, but the first request after a quiet period is slower. Paid instances or an external uptime pinger would remove the delay.

Security, privacy and quality

Security

- Row-level security on every tenant table, enforced by the database.
- Separate restricted and privileged database roles.
- Passwords hashed with bcrypt; minimum length of 12 characters.
- Role checks on sensitive actions; uploads, reports and team changes are admin-only.
- Internal service calls need a shared secret token.
- Expiring, single-report public share links.
- Invitation and reset links are treated as secrets and kept out of logs and API responses.

Privacy

- Emails, phone numbers, card numbers, API-key-like strings and labelled personal fields are redacted before text reaches any model.
- Raw feedback stays in the database and is shown only to authorised members.
- Answers are grounded in the workspace's own data.
- A written PII and LLM policy documents the residual risk.

Quality checks

Check	What it covers
Integration tests (3)	Real-database tests for tenant isolation and public reports, protected routes, and file-scoped analytics including deletion, keyword counts, file-name sources and the pipeline status.
AI service unit tests (7)	Sentiment output validation, internal token check, source-marker parsing, and the rules that accept an honest "I don't know" and reject unsupported citations.
Static analysis	ESLint, TypeScript, Ruff and mypy run on every push.
Supply chain	Dependency audit at high severity and gitleaks secret scanning.
Production build	The full Next.js build runs in CI, which caught a text-encoding mistake before it reached users.
Health checks	Every deployed component exposes a public health endpoint.

Operational guidance

- Monitor `/api/health` from an external uptime service and alert on new production errors.
- Before launch, load-test with a large anonymised workspace and record p50, p95 and p99 latency, database CPU and queue lag.
- Verify after each release that the web role cannot bypass row-level security and that the workers' credential is not present in the web environment.

Challenges, lessons learned and next steps

Challenge	Cause	What was done
Ask LOOP showed an error for good answers	Gemini correctly replied "I don't know", but the strict validator treated any answer without a source marker as a failure.	Treat an honest "I don't know" as a valid answer, accept common marker formats, build citations from the markers used, and add unit tests.
Analysis stuck at 0%	The free AI service had gone to sleep and answered 502 to every worker call.	Retry through cold starts, wake the services together after an upload, and show a "still working" message in the progress panel.
Sentiment trend was a single dot	All rows of an upload share one date, so a line chart had nothing to draw.	Redesign around what the data can say: split, keywords, themes and per-file comparison. Show the time trend only for multi-day data.
Themes appeared very late	Clustering ran only once a week.	Queue clustering a few seconds after embeddings finish; drain all batches per run and write in bulk.
Gemini free-tier rate limit	Embeddings are limited to 100 requests a minute.	Workers pause 45 seconds after a rate limit and resume automatically.
Production build failed	A file was saved with the wrong text encoding.	Fixed the file, verified every changed file, and relied on the CI build as a safety net.
CI could not reach its database	The Postgres service container did not publish its port.	Mapped port 5432 in the workflow.

Lessons learned

- Test AI features against the real model, not assumptions.
- Make honest uncertainty a valid result.
- Design for slow moments, not only fast ones.
- Choose charts to fit the data you actually have.
- Automated checks are cheaper than apologies.

Known limitations

- Free services cold-start (30 to 60 seconds).
- Gemini's free tier rate-limits very large uploads.
- Themes need at least 3 similar items.
- The test email sender only reaches its owner until a domain is verified.
- Keyword extraction removes common filler words but is not a language model.

Next steps

- Run on paid instances or add an uptime pinger to remove cold starts.
- Verify a sending domain in Resend so invitations reach any address.
- Add an "explain this theme" view and export of charts as images.
- Support additional file formats and languages.
- Add automated end-to-end browser tests and error monitoring.

Team, links and API reference

Developers
Sonia
Purnima

Links
Live app: <https://project-loop-two.vercel.app>
Source: <https://github.com/devxjitin/Project-loop>
AI health: <https://loop-ai-service-801c.onrender.com/health>
Workers health: <https://loop-workers.onrender.com/health>

API overview

All endpoints are scoped to the signed-in workspace and need a bearer token unless noted.

Area	Endpoints
Auth	POST /api/auth/signup , /login , /refresh , /password-reset/request , /password-reset/confirm (public)
Uploads	POST /api/ingestions , PUT /api/ingestions/:jobId/upload , GET /api/ingestions , DELETE /api/ingestions/:jobId (also removes the file's feedback and analytics)
Feedback	GET /api/feedback (filters: jobId, sentiment, themeId, q) , GET /api/feedback/:id , GET /api/themes
Analytics	GET /api/analytics/trend , /themes , /files , /keywords , /status , each with an optional ? jobId= to scope to one file
Ask LOOP	POST /api/qa/answer , POST /api/qa/retrieve , GET /api/qa/sessions/:id , GET /api/qa/warm
Reports	POST /api/reports , GET /api/reports , POST /api/reports/:id/share , GET /api/reports/:id/pdf , PUT /api/report-schedule , GET /api/public/reports/:token (public)
Team	GET/POST /api/members , PATCH/DELETE /api/members/:id , POST /api/invitations/accept
Health	GET /api/health (public)

Repository layout

apps/web	Next.js app: pages, API routes, workers, server libraries and tests
apps/ai-service	FastAPI service wrapping Gemini and HDBSCAN
packages/database	SQL migrations and the migration runner
infra , docs	Database role script, deployment notes, operations guide and PII policy

Thank you. Built by Sonia and Purnima.